

WORKSHOP

การพัฒนา Web Application ด้วย Google Antigravity

กรณีศึกษา : Funding Hub — ระบบจัดการข้อมูลทุนวิจัย

ศักดิ์ดา หอมหวล

เป้าหมาย

เมื่อจบวันนี้ ผู้เข้าอบรมจะทำได้ด้วยตนเอง

1

เข้าใจโครงสร้างเว็บแอปพลิเคชัน

แยกออกว่า Frontend / Backend / Database ทำหน้าที่ต่างกันอย่างไร

2

สั่งงาน AI Agent ด้วย Antigravity

รู้จัก Editor, Agent, Artifacts และ Browser Subagent

3

เขียนสเปกที่ AI ทำตามได้จริง

แปลงโจทย์เป็นเอกสารสเปกไฟล์เดียว พร้อมเกณฑ์ยอมรับที่วัดผลได้

4

สร้างเว็บที่ใช้งานได้จริงและนำขึ้นระบบ

ครบ 4 หน้า เชื่อมฐานข้อมูลจริง และเปิดจาก URL สาธารณะ

กำหนดการ 3 ชั่วโมง 30 นาที

บรรยาย 110 นาที · ปฏิบัติ 100 นาที — แบ่งเป็น 3 แล็บ ต่อเนื่องกันเป็นเรื่องเดียว

10 น.	เปิดการอบรม / ภาพรวม	20 น.	ออกแบบ UI : สไตล์ สี ฟอนต์
20 น.	พื้นฐาน Web Application	15 น.	LAB B · ตอนที่ 2 — Design System
10 น.	LAB A · เตรียมเครื่องและเปิดโปรเจกต์	10 น.	ฐานข้อมูลด้วย Supabase
15 น.	รู้จัก Google Antigravity	10 น.	LAB B · ตอนที่ 3 — ฐานข้อมูล
5 น.	LAB A · วางบล็อกแรกของสเปก	10 น.	กลยุทธ์สั่งงาน AI Agent
20 น.	แกะโจทย์ให้เป็นสเปก	30 น.	LAB C · สั่ง Antigravity สร้างแอป
20 น.	LAB B · ประกอบสเปก ตอนที่ 1	15 น.	ทดสอบ · Deploy · สรุป

01

พื้นฐาน Web Application

รู้ว่าเรากำลังสร้างอะไร ก่อนจะสั่งให้ AI สร้างให้

เว็บไซต์ ต่างจาก เว็บแอปพลิเคชัน อย่างไร

เว็บไซต์ (Website)

- ผู้ใช้อ่านอย่างเดียว
- เนื้อหาคงที่ แก้ต้องแก้ไฟล์
- ไม่จำเป็นต้องมีฐานข้อมูล
- ตัวอย่าง : หน้าประชาสัมพันธ์คณะ

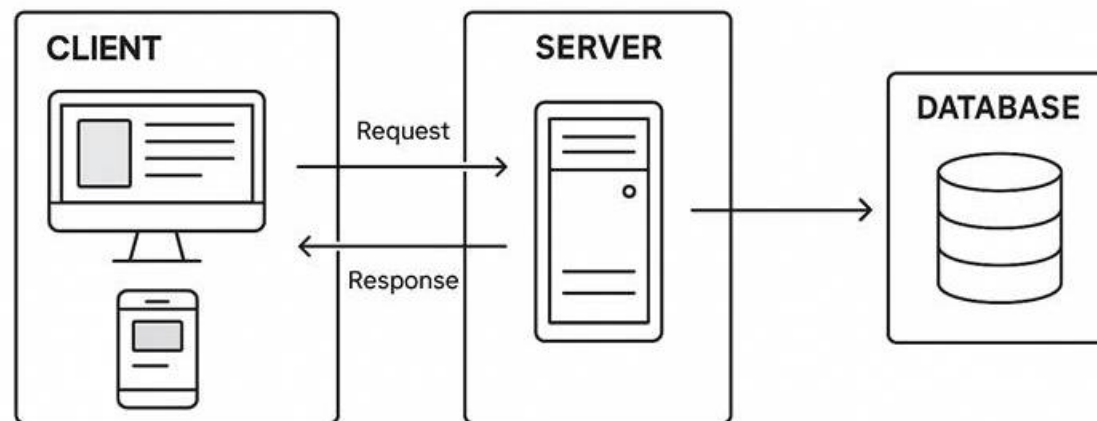
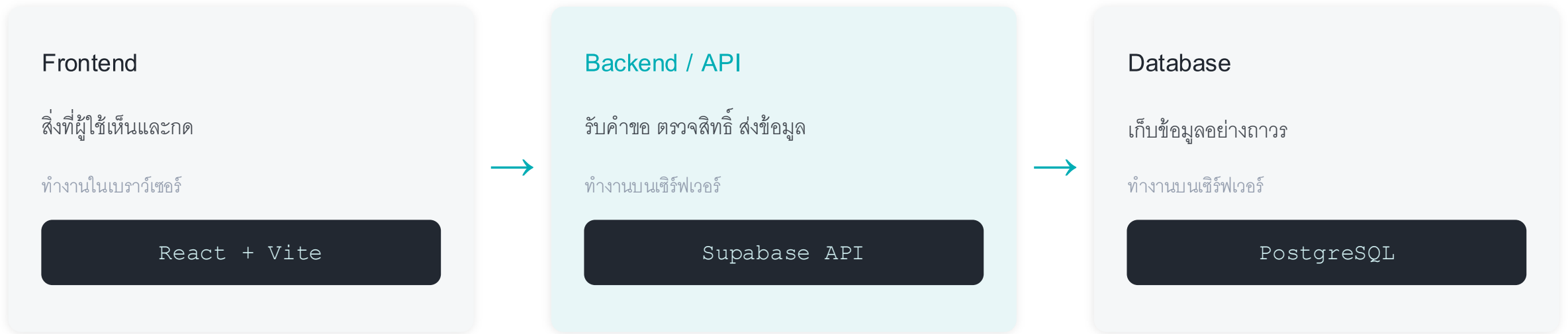
เว็บแอปพลิเคชัน (Web Application)

- ผู้ใช้ป้อน แก้ไข และลบข้อมูลได้
- เนื้อหาเปลี่ยนตามข้อมูลในระบบ
- ต้องมีฐานข้อมูลเสมอ
- ตัวอย่าง : Funding Hub ที่เราจะสร้าง

Funding Hub เป็นเว็บแอปพลิเคชัน เพราะมีการ เพิ่ม / อ่าน / แก้ไข / ลบ ข้อมูล — เรียกรวมว่า CRUD

สถาปัตยกรรม 3 ชั้นของเว็บแอปพลิเคชัน

ทุกเว็บแอปในโลกนี้ประกอบด้วยสามชั้นนี้เสมอ ไม่ว่าจะใช้เทคโนโลยีอะไร



ภาษาและเครื่องมือฝั่ง Frontend

HTML

โครงสร้างของหน้า — เปรียบเหมือนโครงกระดูก

ปุ่ม ตาราง ฟорм อยู่ตรงไหน

CSS

หน้าตาและการจัดวาง — เปรียบเหมือนเสื้อผ้า

สี ฟอนต์ ระยะห่าง การตอบสนองต่อขนาดจอ

JavaScript

พฤติกรรม — เปรียบเหมือนกล้ามเนื้อ

กดปุ่มแล้วเกิดอะไร ดึงข้อมูลจากฐานข้อมูลอย่างไร

แล้ว React กับ Vite คืออะไร

- React คือไลบรารีที่ช่วยแบ่งหน้าเว็บออกเป็นชิ้นส่วนเล็ก ๆ (Component) เช่น ปุ่ม การ์ดทุน แล้วนำกลับมาใช้ซ้ำได้
- Vite คือเครื่องมือรวมและรันโค้ด ทำให้แก้ไขไฟล์แล้วเห็นผลบนเบราว์เซอร์ทันทีโดยไม่ต้องกดรีเฟรช

CRUD และพื้นฐานฐานข้อมูล

คำสี่คำที่อธิบายได้ทุกอย่างที่ระบบข้อมูลทำ

การกระทำ	HTTP	คำสั่ง SQL	ในระบบของเรา
Create — สร้าง	POST	insert	เพิ่มทุนวิจัยใหม่
Read — อ่าน	GET	select	แสดงรายการทุน
Update — แก้ไข	PATCH	update	แก้ไขข้อมูลทุนเดิม
Delete — ลบ	DELETE	delete	ลบทุนออกจากระบบ

ตารางในฐานข้อมูล

- ตาราง (Table) — เก็บข้อมูลชนิดเดียวกัน เช่น ตารางเดียวกัน เช่น ตาราง grants เก็บทุนวิจัยวิจัย
- คอลัมน์ (Column) — ฟیلด์ข้อมูล เช่น ชื่อทุน แหล่งทุน กำหนดปีได้รับ
- แถว (Row) — ทุน 1 รายการ
- Primary Key — รหัสประจำแถวที่ไม่ซ้ำกัน ใช้ระบุว่าจะแก้ไขหรือลบแถวไหน
- ชนิดข้อมูล — text, date, uuid บอกว่าคอลัมน์นั้นเก็บอะไรได้

Supabase และชุดเทคโนโลยีของเวิร์กช็อป

Supabase คืออะไร

บริการฐานข้อมูลสำเร็จรูปบนคลาวด์ (Backend as a Service)
สร้างตารางเสร็จ ได้ API ใช้ทันที ไม่ต้องเขียนเซิร์ฟเวอร์เอง

- 1 PostgreSQL ฐานข้อมูลมาตรฐานอุตสาหกรรม
- 2 API อัตโนมัติ สร้างให้ทันทีจากโครงสร้างตาราง
- 3 RLS กำหนดสิทธิ์ว่าใครทำอะไรได้บ้าง
- 4 Storage / Auth มีให้ใช้ แต่ไม่ใช้ในเวิร์กช็อปนี้

ชุดเทคโนโลยีที่เราจะใช้

Vite + React	สร้างหน้าเว็บ
Tailwind CSS	จัดสไตล์ตาม Design System
Supabase	ฐานข้อมูลและ API
GitHub + Pages	เก็บโค้ดและนำขึ้นระบบฟรี
Antigravity	AI Agent ที่เขียนโค้ดให้เรา

สิ่งที่ต้องทำ

- 1 ตรวจสอบว่ามี Node.js, Git และ Antigravity IDE แล้ว
- 2 สร้างโฟลเดอร์ fundinghub แล้วเปิดใน Antigravity
- 3 สร้างไฟล์เปล่าชื่อ SPEC_FundingHub.md ที่รากโปรเจกต์
- 4 เปิดไฟล์ SPEC_BLOCKS.md ที่แจกให้ ให้อีกหน้าต่างหนึ่ง
- 5 หาให้เจอ 4 ส่วน : Editor / Agent / Artifacts / Browser

```
node -v  
npm -v  
git --version
```

เกณฑ์ผ่าน

- ทุกคำสั่งขึ้นเลขเวอร์ชัน ไม่ขึ้น command not found
- เปิดโฟลเดอร์ fundinghub ใน Antigravity ได้
- มีไฟล์เปล่า SPEC_FundingHub.md รออยู่

02

รู้จัก Google Antigravity

เครื่องมือที่เราจะใช้สั่งงานตลอดทั้งวัน

Google Antigravity คืออะไร

- สภาพแวดล้อมการพัฒนาแบบ agent-first ของ Google เปิดตัวเดือนพฤศจิกายน 2568 พัฒนาต่อจาก VS Code
- แนวคิดหลักคือเรามอบหมายงานระดับ “สิ่งที่ยากได้” ให้ AI Agent แทนการพิมพ์โค้ดทุกบรรทัดเอง
- ปี 2569 มี Antigravity 2.0 เป็นแอปแยกสำหรับสั่งงาน Agent หลายตัวพร้อมกัน พร้อม CLI และ SDK



เวิร์กช็อปนี้ใช้ Antigravity IDE ไม่ใช่ Antigravity 2.0

เพราะผู้เริ่มต้นควรเห็นทั้งไฟล์โค้ด หน้าต่างคำสั่ง และเบราวเซอร์ทดสอบอยู่ในหน้าจอเดียวกัน จะได้เข้าใจว่า Agent กำลังทำอะไรกับโปรเจกต์ของเรา

องค์ประกอบสำคัญ 4 ส่วนที่ต้องรู้จัก

เปิดโปรแกรมแล้วให้หาส่วนนี้ให้เจอก่อน

1

Editor

หน้าต่างแก้ไขไฟล์โค้ดตามปกติ มีการเติมโค้ดอัตโนมัติด้วยปุ่ม Tab เราใช้ดูว่า Agent เขียนอะไรลงไป

2

Agent Manager

ช่องสนทนาสำหรับสั่งงาน และเฝ้าติดตามว่า Agent กำลังทำงานไหนอยู่ สั่งหลายงานพร้อมกันได้

3

Artifacts

ผลงานระหว่างทางที่ Agent สร้าง เช่น แผนงาน รายการสิ่งที่จะทำ ผลทดสอบ เราสามารถคอมเมนต์แก้ไขได้

4

Browser Subagent

เบราว์เซอร์ที่ Agent เปิดและควบคุมเอง เพื่อกดปุ่ม กรอกฟอร์ม และถ่ายภาพหน้าจอมายืนยันว่าใช้งานได้จริง

Vibe Coding กับ Spec-driven Development

สองวิธีสั่งงาน AI ที่ให้ผลลัพธ์ต่างกันมาก

Vibe Coding

พิมพ์บอกสั้น ๆ ตามความรู้สึก แล้วดูว่า AI ให้อะไรมา

- ได้ผลเร็วมากในช่วงแรก
- สั่งคำเดิมสองครั้ง ได้ผลไม่เหมือนกัน
- AI เต็มพีเจอร์ที่เราไม่ได้ขอ
- แก่จุดหนึ่ง พังอีกจุดหนึ่ง
- ไม่มีเกณฑ์บอกว่างานเสร็จหรือยัง

Spec-driven Development

เขียนข้อกำหนดให้ครบก่อน แล้วสั่งงานโดยอ้างหมายเลขหัวข้อ

- ผลลัพธ์คงเส้นคงวา ทำซ้ำได้
- AI ไม่สร้างเกินขอบเขตที่เขียนไว้
- แก้สเปกทีเดียว งานตามทั้งระบบ
- มีเกณฑ์ยอมรับไว้ตรวจว่าผ่านหรือไม่
- prompt สั้นลงเหลือ 3–5 บรรทัด

วันนี้เราใช้ Spec-driven : เขียน FundingHub.md เพียงไฟล์เดียว แล้ว prompt ทุกอันชี้ไปที่หมายเลขหัวข้อในไฟล์นั้น

โครงสร้างของ Prompt ที่ใช้งานได้จริง

prompt ที่ดีไม่ใช่ prompt ที่ยาว แต่คือ prompt ที่ระบุครบและตรวจสอบได้

1

งานที่ต้องทำ

บอกให้ชัดว่าทำขั้นไหน ขั้นไหน

2

บริบทที่ต้องอ่าน

ระบุชื่อไฟล์และหมายเลขหัวข้อ

3

ข้อจำกัด

อะไรห้ามแตะ อะไรห้ามเพิ่ม

4

วิธียืนยันผล

ให้ทดสอบและถ่ายภาพหน้าจอมาให้ดู

ตัวอย่าง prompt จริงที่จะใช้ใน LAB C

```
ทำชั้น L3 ตาม SPEC_FundingHub.md §11.3  
สร้างหน้า /grants ให้ครบตาม §7.2  
ใช้เฉพาะ token สีและตัวอักษรใน §4  
ทดสอบด้วย Browser แล้วถ่าย screenshot
```

สี่บรรทัดนี้แทนคำอธิบายกว่า 300 บรรทัด เพราะรายละเอียดทั้งหมดอยู่ในไฟล์สเปกแล้ว ไม่ต้องพิมพ์ซ้ำและไม่มีโอกาสพิมพ์ตกหล่น

สิ่งที่ต้องทำ

- 1 เปิด SPEC_BLOCKS.md หาบล็อกที่ 1 (§0 กติกา + §1 ขอบเขต)
- 2 คัดลอกทั้งบล็อก วางลงใน SPEC_FundingHub.md แล้วบันทึก
- 3 วาง prompt ด้านขวาในช่อง Agent
- 4 อ่านคำตอบร่วมกัน — Agent ต้องรู้ขอบเขตแล้ว

อ่านไฟล์ SPEC_FundingHub.md
สรุปเป็นภาษาไทยว่า ระบบนี้มีกี่หน้า
อะไรบ้าง และมีอะไรบ้างที่ห้ามสร้าง
ยังไม่ต้องเขียนโค้ด

คำตอบที่ถูกต้อง

- 4 หน้า : Dashboard / ทุนวิจัย / เพิ่มทุน / Admin
- ห้ามสร้าง AI / LINE / Proposal / Project Tracking / ระบบบล็อกอื่น

03

แกะโจทย์ให้กลายเป็นสเปก

ขั้นตอนที่สำคัญที่สุดของวันนี้ และเป็นขั้นที่คนข้ามบ่อยที่สุด

โจทย์ตั้งต้น : Funding Hub

ข้อความที่ผู้ใช้เขียนมาให้เรา — สั้น กระชับ แต่ยังสั่ง AI ตรง ๆ ไม่ได้

4 หน้าเท่านั้น

Dashboard · รายการทุน · เพิ่ม/แก้ไข · Admin

ค้นหาและกรอง

ค้นหาชื่อทุน และกรองตามสถานะ

ฐานข้อมูล

ใช้ Supabase ชื่อโปรเจกต์ fundinghub พร้อม CRUD

KPI 4 ช่อง

เปิดรับ · ใกล้เคียงเขต · ปิดรับแล้ว · ทั้งหมด

ฟอร์ม 7 ช่อง

ชื่อทุน แหล่งทุน Theme Deadline URL หมายเหตุ สถานะ

ข้อมูลตัวอย่าง 3 รายการ

TREE Fund · Belmont Forum CRA · ASEA-UNINET

ประโยคที่มีค่าที่สุดในโจทย์ทั้งหมด

“ไม่ต้องสร้าง AI, LINE, Proposal, Project Tracking หรือระบบซับซ้อนอื่น” — สิ่งที่ทำให้ AI สร้างงานเกินจริงมากที่สุด คือการไม่บอกว่าอะไรที่ไม่ต้องทำ

04

ออกแบบ UI ก่อนสั่ง AI

กำหนดสไตล์ สี และฟอนต์ด้วยตัวเอง เพื่อไม่ให้งานของทุกคนหน้าตาเหมือนกัน

ทำไมต้องกำหนด Design System ก่อน

ถ้าไม่กำหนด

- AI เลือกสีน้ำเงินและฟอนต์สากลให้ทุกครั้ง
- งานของผู้เข้าอบรมทุกคนหน้าตาเหมือนกันหมด
- สั่งแก้หน้าหนึ่ง สีเพี้ยนไปอีกหน้าหนึ่ง
- ภาษาไทยแสดงผลเพี้ยนเพราะฟอนต์ไม่รองรับ

ถ้ากำหนดไว้ใน §4

- ทุกหน้าใช้สีและฟอนต์ชุดเดียวกันโดยอัตโนมัติ
- เปลี่ยนสีทั้งระบบด้วยการแก้ไฟล์สเปกทีเดียว
- งานออกมาเป็นเอกลักษณ์ของเราเอง
- ภาษาไทยอ่านง่าย เพราะเลือกฟอนต์ที่รองรับ

สามสิ่งที่ต้องเลือกใน LAB 3

1

สไตล์

uistyleguide.com

2

จานสี

colorhunt.co

3

ฟอนต์

fonts.google.com

เลือกสไตล์การออกแบบ : uistyleguide.com

เก็บรวมสไตล์ UI กว่า 67 แบบ พร้อมตัวอย่างที่ทดลองได้จริงและคำแนะนำการใช้ CSS

Minimalism	อ่านข้อมูลง่าย เหมาะกับงานวิชาการและราชการ	แนะนำ
Flat Design	เรียบ ทำเร็ว รองรับหลายขนาดจอได้ดี	ใช้ได้
Soft UI	นุ่มนวลสบายตา แต่ต้องระวังความคมชัดของตัวอักษร	ระวัง
Glassmorphism	สวยแต่ทำให้อ่านตัวเลข KPI ยาก	ไม่แนะนำ

วิธีจดสไตล์ลงสเปก

เขียนหลักการที่ “มองเห็นได้จริง” 3-5 ข้อ

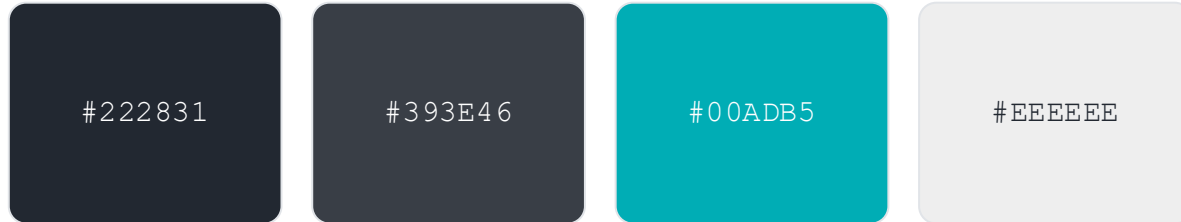
อย่าเขียนว่า
“ทำให้ดูโมเดิร์น”

ให้เขียนว่า
“การ์ดพื้นขาว มุมมน 12px
เงาอ่อน ไม่มีเส้นขอบหนา”

เลือกงานสี : colorhunt.co

เลือกชุดสี 4 สีที่ผ่านการจับคู่มาแล้ว แทนการเดาสีเอง

งานสีที่เวิร์กช็อปนี้ใช้เป็นค่าเริ่มต้น



จับคู่สีเข้ากับหน้าที่ ด้วยหลัก 60 – 30 – 10

สีอ่อนที่สุด	พื้นหลังหน้าเว็บ	60%
สีเข้มที่สุด	ตัวอักษรและแถบเมนู	30%
สีที่สดที่สุด	ปุ่มหลักและจุดเน้นเท่านั้น	10%

ตรวจความคมชัดเสมอ

สีตัวอักษรกับสีพื้นหลังต้องมีอัตราส่วนความคมชัดอย่างน้อย 4.5 : 1

น้อย 4.5 : 1

ตรวจได้ที่ webaim.org/resources/contrastchecker

สีสถานะ — ไม่ต้องเปลี่ยน

เปิดรับ

ใกล้หมดเขต

ปิดรับแล้ว

เขียว – เหลือง – เทา เป็นสีสื่อความหมายที่คนทั่วโลกเข้าใจตรงกัน การเปลี่ยนจะทำให้ผู้ใช้สับสน

เลือกฟอนต์ : Google Fonts

สำคัญมาก — ต้องกรอง Language = Thai เท่านั้น มิฉะนั้นสระและวรรณยุกต์จะแสดงผลเพี้ยน

Sarabun	ทางการ อ่านง่าย เป็นกลาง	เอกสารวิชาการและราชการ
IBM Plex Sans Thai	สมัยใหม่ ตัวเลขคมชัด	ระบบข้อมูลและเดสก์บอร์ด
Noto Sans Thai	กลาง ๆ ปลอดภัย	ใช้ได้กับทุกงาน
Prompt	โมเดิร์น เรขาคณิต	เว็บองค์กรและงาน ปรส.
Kanit	หนักแน่น เด่นชัด	หัวข้อใหญ่ ไม่เหมาะเนื้อหายาว

กฎการใช้ฟอนต์ไทย

- 1 โหลดแค่ 2-3 น้ำหนัก (400 / 600 / 700) โหลดหมดทำให้เว็บช้า
- 2 ตั้ง line-height อย่างน้อย 1.6 เพราะภาษาไทยมีสระบนและล่าง ถ้าบรรทัดชิดจะชนกัน
- 3 ห้ามใช้ text-transform uppercase เพราะภาษาไทยไม่มีตัวพิมพ์ใหญ่

05

ฐานข้อมูลและการสร้างแอป

จากสเปกบนกระดาษ สู่ระบบที่ใช้งานได้จริง

สร้างฐานข้อมูลด้วย Supabase

สามขั้นตอน ใช้เวลารวมไม่เกิน 15 นาที

1

สร้างโปรเจกต์

ตั้งชื่อ fundinghub เลือก region Singapore และตั้งรหัสผ่านฐานข้อมูลเอง

2

รัน SQL สร้างตาราง

คัดลอกคำสั่งจาก จาก AI ที่ให้มา ครอบคลุมตาราง ดัชนี trigger และ RLS policy

3

ใส่ข้อมูลตัวอย่างและคัดลอกกุญแจ

รัน INSERT จาก AI ที่ให้มา แล้วคัดลอก Project URL กับ anon key มาไว้ใน .env.local

RLS — Row Level Security คืออะไร

กติกาในฐานข้อมูลที่กำหนดว่า “ใครทำอะไรกับแถวไหนได้บ้าง” เวิร์กช็อปนี้เปิดสิทธิ์ให้ทุกคนเพื่อความง่าย แต่ระบบใช้งานจริงต้องผูกกับผู้ใช้ที่ล็อกอิน

กฎเหล็กเรื่องความลับ

- ใช้ได้เฉพาะ anon key เท่านั้น
- service_role key ห้ามปรากฏในฝั่งหน้าเว็บเด็ดขาด เพราะข้าม RLS ได้ทั้งหมด
- รหัสผ่านฐานข้อมูลห้ามเขียนลงไฟล์ใด ๆ ในโปรเจกต์ และห้ามส่งให้ AI
- ใส่ .env.local ไว้ใน .gitignore เสมอ ก่อน push ให้รัน git status ตรวจซ้ำ

สังเกตว่าไจท์ต้นฉบับมีบรรทัด password: xxx — นี่คือตัวอย่างของสิ่งที่ไม่ควรเขียนไว้ในเอกสารที่แชร์กัน

ข้อจำกัดของ AI และความรับผิดชอบของเรา

สิ่งที่ต้องจำให้ขึ้นใจเมื่อกลับไปทำงานจริง

1

AI ไม่รู้ว่าจะไรถูก ถ้าเราไม่บอก

มันทำตามสิ่งที่เขียนไว้ ไม่ใช่สิ่งที่เราคิดในใจ ความคลุมเครือทุกจุดจะกลายเป็นการเดา

2

โค้ดที่ได้ยังเป็นความรับผิดชอบของเรา

ต้องอ่าน ต้องทดสอบ ต้องเข้าใจก่อนนำไปใช้จริง โดยเฉพาะระบบที่มีข้อมูลของคนอื่น

3

ความลับห้ามอยู่ในโค้ด

กุญแจ รหัสผ่าน และข้อมูลส่วนบุคคล ต้องอยู่นอกโปรเจกต์เสมอ

4

ระบบจริงต้องมีสิทธิ์ผู้ใช้อย่างจริง

RLS แบบเปิดทุกสิทธิ์ใช้ได้เฉพาะการฝึกอบรม ระบบที่มีข้อมูลจริงต้องผูกกับผู้ใช้ที่ล็อกอินและคำนึงถึง PDPA

สรุปสิ่งที่ได้เรียนรู้

1 เขียนสเปกก่อน แล้วค่อยสั่ง

SPEC ไฟล์เดียว + prompt สั้นที่อ้างหมายเลขหัวข้อ

2 ออกแบบก่อนสร้าง

สไตล์จาก uistyleguide · สีจาก colorhunt · ฟอนต์จาก Google Fonts

3 สร้างทีละชั้น ตรวจสอบทีละชั้น

L1 ถึง L6 และตรวจด้วยเกณฑ์ยอมรับที่วัดผลได้

4 งานเสร็จเมื่อคนอื่นเปิดใช้ได้

ทดสอบ 3 ระดับ แล้วนำขึ้น GitHub Pages

ขอบคุณครับ · ช่วงถาม-ตอบ

ไฟล์ประกอบ : SPEC_FundingHub.md · LAB_FundingHub_Antigravity.md